

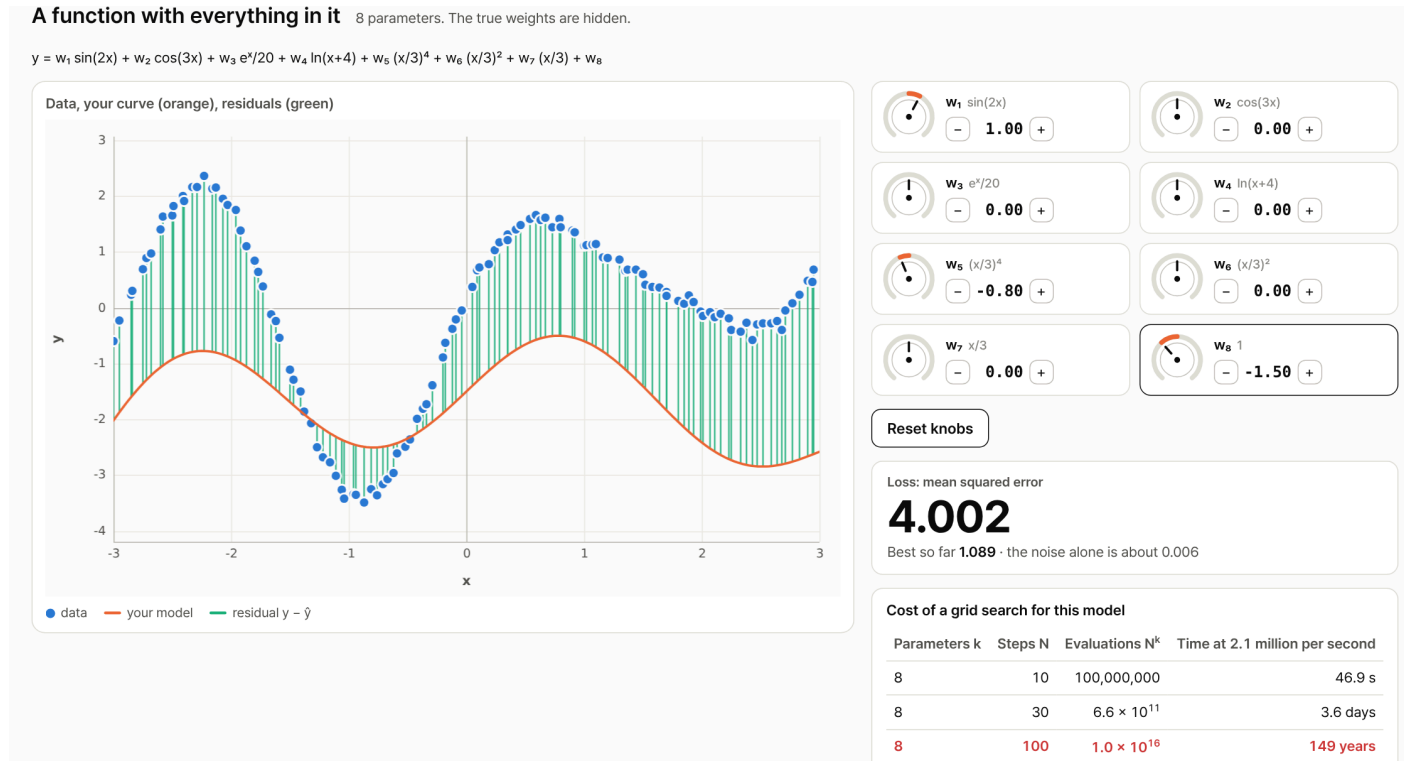
Data & AI

4. Gradient Descent

Dasaem Jeong · Art & Technology, Sogang University

Last Time

Eight knobs, one hidden function



- By hand, the loss stopped well above the bottom
- A grid over every combination would take 100 years
- The machine reached the bottom in a few seconds

Today

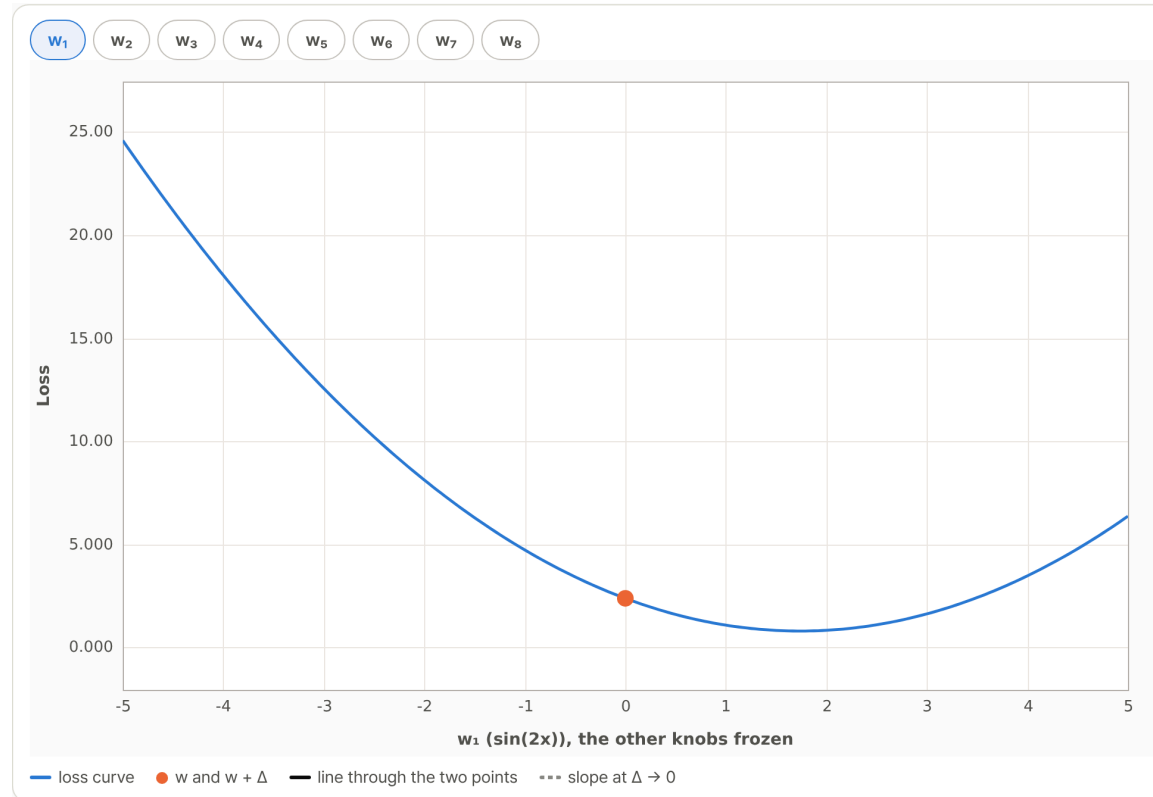
jdasam.github.io/mas1004/demos/lecture04-gradient-descent

- Measure how the loss changes when one knob moves a little
- Turn every knob a small step downhill, and repeat
- See what the step size and the starting point change

The slope of a curve

One Knob at a Time

w_1 , the other seven knobs frozen

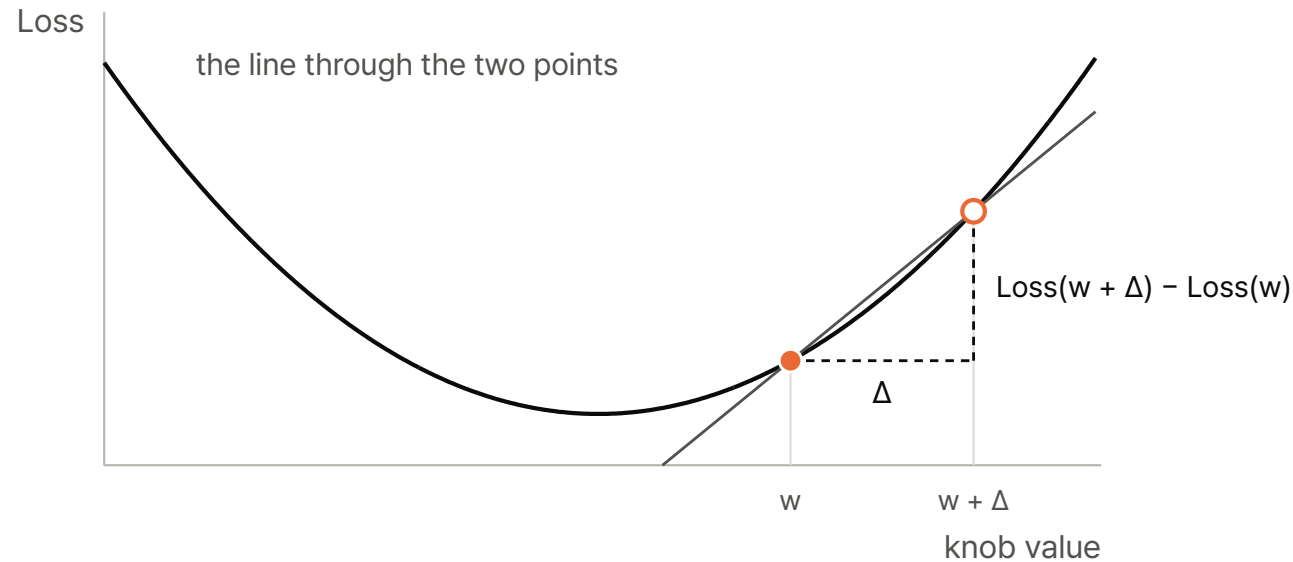


- Horizontal axis: the value of w_1 , not the input x
- Loss as a function of one knob: a curve with a bottom

Slope from Two Points

Nudge the knob by Δ and measure again

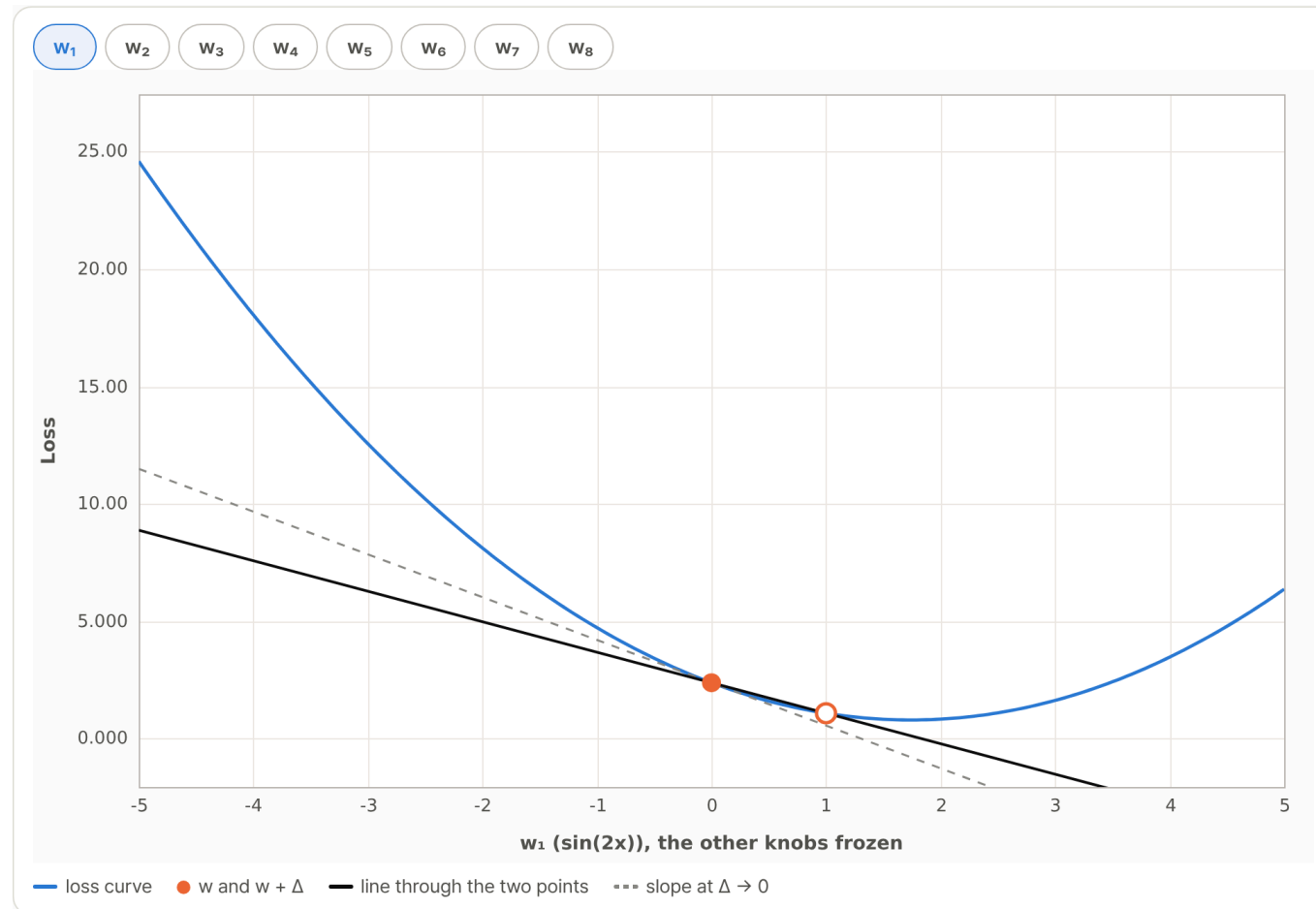
$$\text{slope} \approx \frac{\text{Loss}(w + \Delta) - \text{Loss}(w)}{\Delta}$$



- Two loss computations: one here, one a little to the right

$$\Delta = 1$$

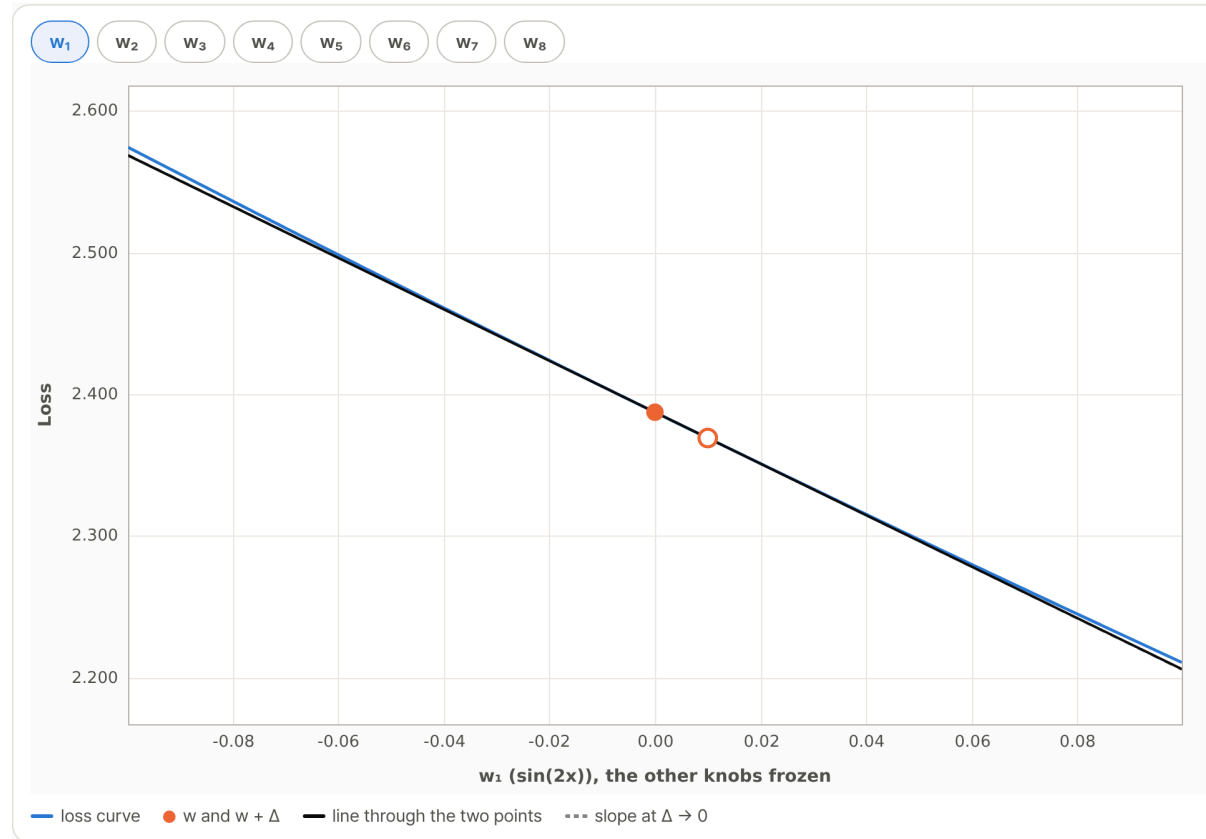
Demo, tab 1



- The line through the two points misses the curve between them

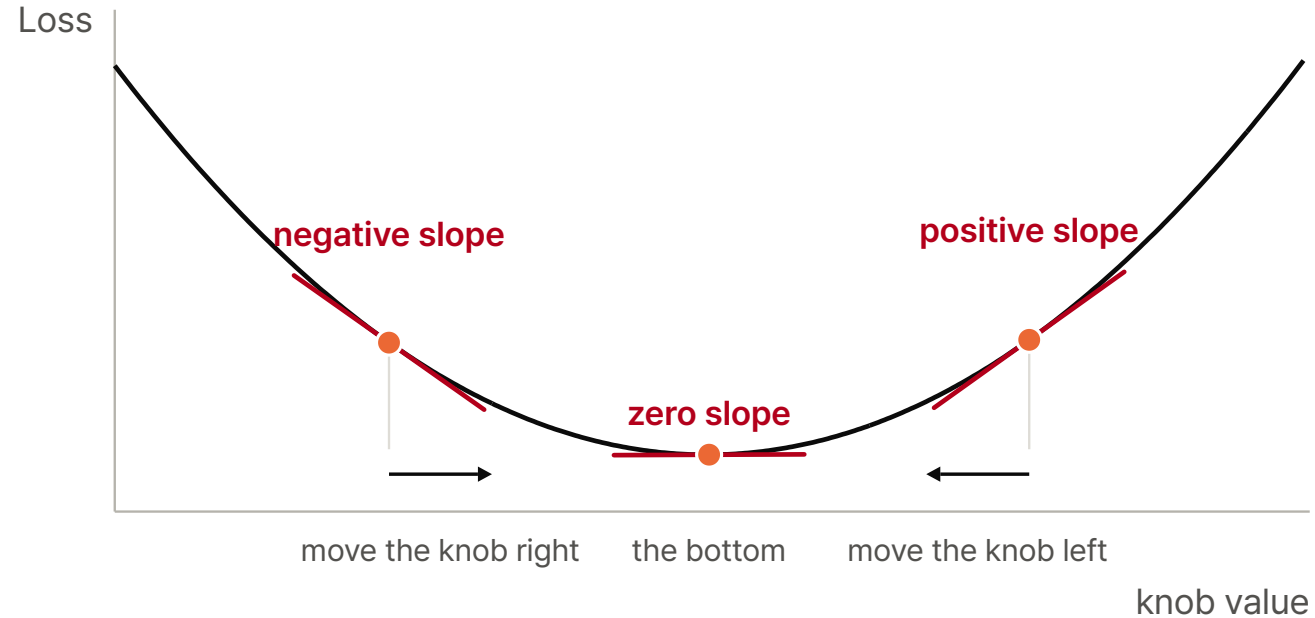
$$\Delta = 0.01$$

Demo, tab 1, zoomed in



- As Δ shrinks, the slope settles to one number: the slope at this point
- That number is called the derivative

What the Sign Tells



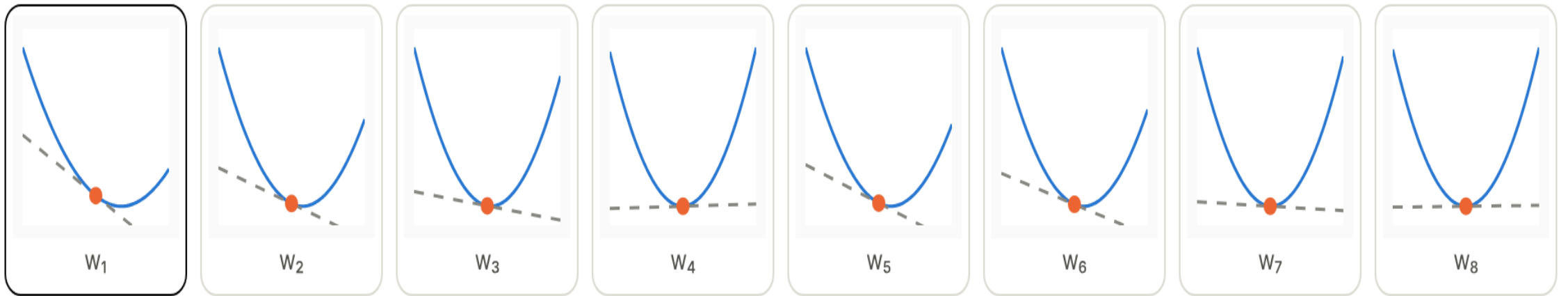
- Positive slope: move the knob left. Negative slope: move it right
- Slope zero: the bottom, or at least a flat spot
- The size of the slope: how steep. It sets the step size in the next section

Many knobs

Freeze the Others

Eight knobs, eight curves

All eight knobs, each with the other seven frozen



- Each curve freezes the other seven knobs at their current values
- The slope of each curve is that knob's partial derivative

The Gradient

Demo, tab 2, "Measure the gradient"

The gradient: one slope per knob
Loss now: **2.387** $\Delta = 0.001$

Knob	Value	Loss at $w + \Delta$	Slope	
$w_1 \sin(2x)$	0.00	2.385	-1.821	
$w_2 \cos(3x)$	0.00	2.387	-0.715	
$w_3 e^x/20$	0.00	2.387	-0.042	
$w_4 \ln(x+4)$	0.00	2.387	+0.131	
$w_5 (x/3)^4$	0.00	2.387	-0.183	
$w_6 (x/3)^2$	0.00	2.387	-0.247	
$w_7 x/3$	0.00	2.387	-0.048	
$w_8 1$	0.00	2.387	+0.031	

Largest |slope|: w_1 . This knob changes the loss the most right now.

- The list of eight slopes at the current position
- Nine loss computations: one here, one per nudged knob

Reading the Table

Values from the demo at the starting position, every knob at 0

Knob	Slope	
w_1	-1.821	matters the most right now
w_2	-0.715	
w_3	-0.042	barely matters right now
w_4	+0.131	
w_5	-0.183	
w_6	-0.247	
w_7	-0.048	barely matters right now
w_8	+0.031	barely matters right now

- Move the knobs and the whole table changes

Gradient descent

The Rule

One step of w_1 from the starting position

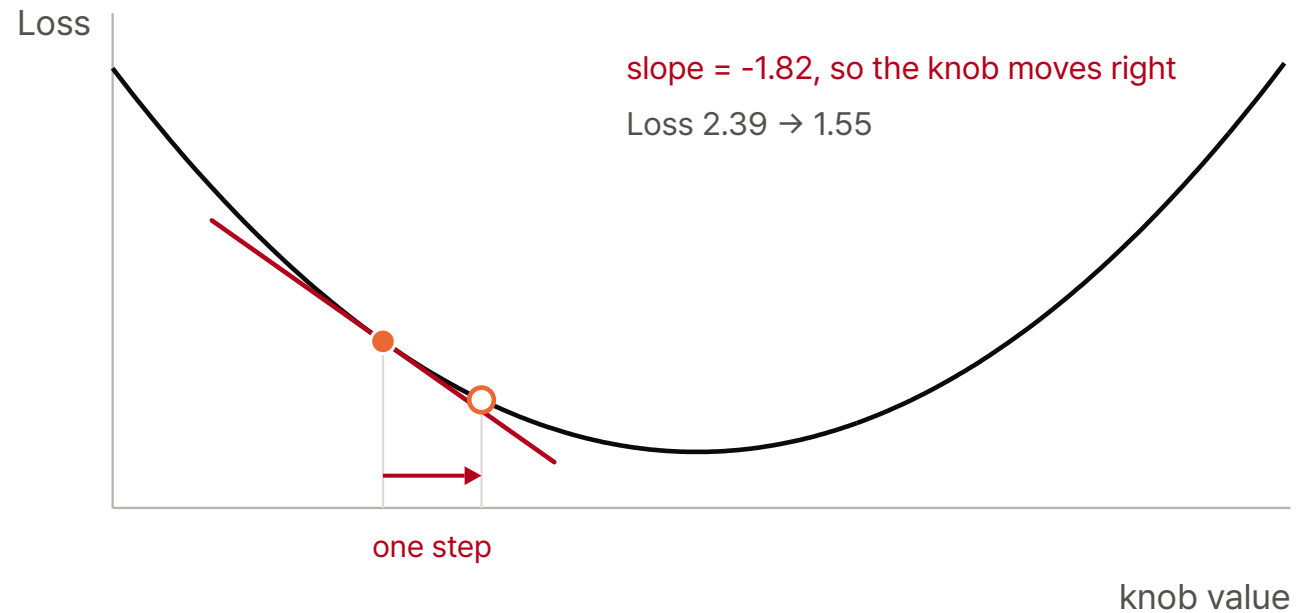
$w \leftarrow w - lr \times \text{slope}$, for every knob at once

$w_1 = 0.00$, slope = -1.821 , $lr = 0.3$: $0.00 - 0.3 \times (-1.821) = 0.546$

- Move each knob against its own slope
- The step is $lr \times \text{slope}$: long where the curve is steep, short near the bottom
- lr , the learning rate: a number we choose
- The rule uses only the slope where it stands

One Step

The same numbers on the curve of w_1



- The knob moves against the slope, and the loss comes down

Two Buttons

Demo, tab 3

Measure the gradient

Take a step

$w \leftarrow w - lr \times \text{slope}$

Run

Reset

1/frame

10/frame

100/frame

step 2, loss 0.876, learning rate 0.3
Loss computed: **18** times

- Measure the gradient, then take a step
- After the step, the old gradient is wrong: measure again

Why Measure Again

The gradient at the new position

Gradient at this position

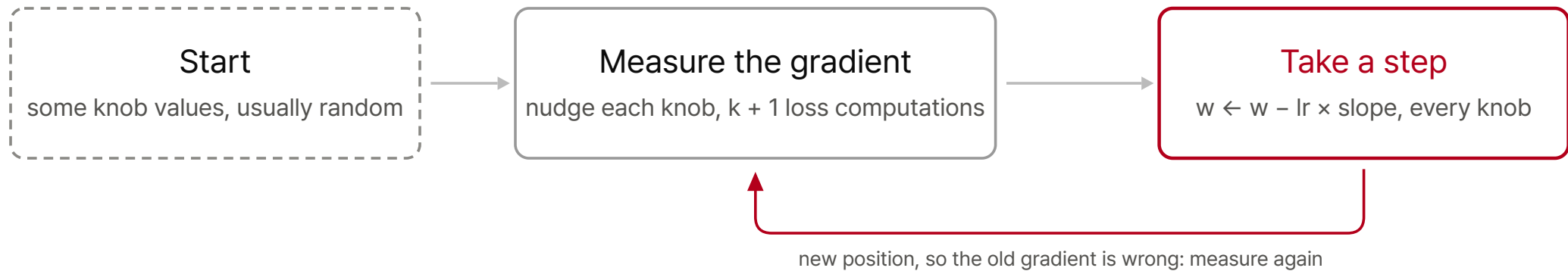
Loss now: **0.876** $\Delta = 0.001$

Knob	Value	Slope before the step	Slope now
$w_1 \sin(2x)$	0.92	-1.244	-0.891
$w_2 \cos(3x)$	0.37	-0.512	-0.359
$w_3 e^x/20$	0.05	-0.120	-0.116
$w_4 \ln(x+4)$	-0.01	-0.090	+0.048
$w_5 (x/3)^4$	0.11	-0.171	-0.125
$w_6 (x/3)^2$	0.14	-0.227	-0.152
$w_7 x/3$	0.09	-0.240	-0.293
$w_8 1$	-0.01	0.004	+0.160

Largest |slope|: w_1 . This knob changes the loss the most right now.

- Same knob, different position, different slope

The Loop

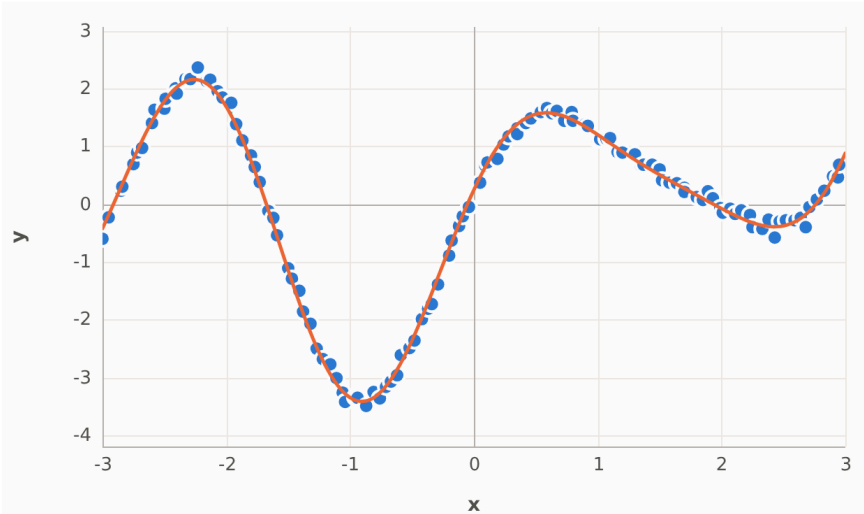


- Start at some knob values, usually random. Then measure, step, measure, step
- Stop when the slopes are all near zero, or after a fixed number of steps

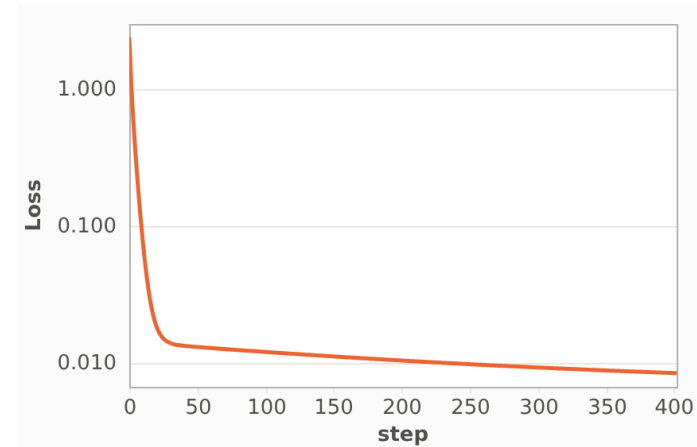
Run

Demo, tab 3, "Run"

Data (blue), your curve (orange), residuals (green)



Loss against step

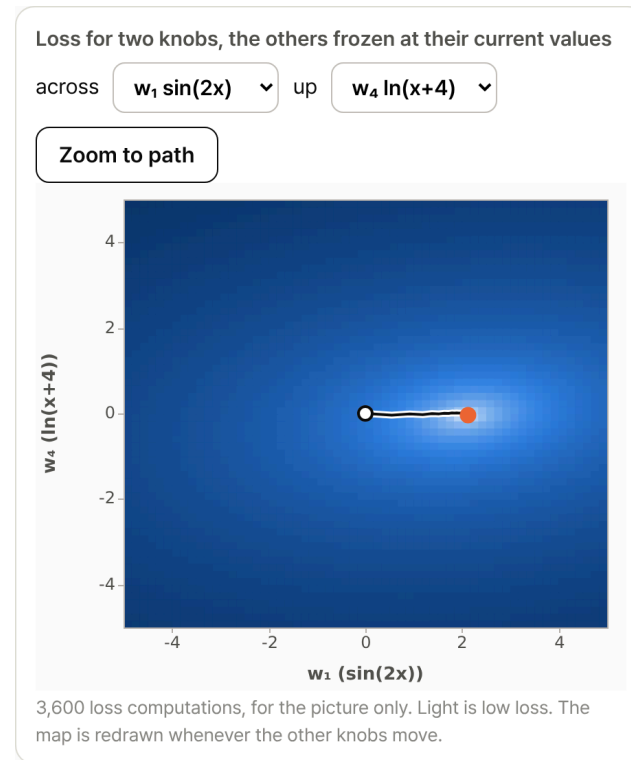


Earlier runs stay in grey, up to three. Reset returns the knobs to the start and keeps them.

- Loss against step: fast at first, slower near the bottom
- Loss computed about 3,600 times in total. The grid needed 10^{16}
- Reset, turn the knobs anywhere, Run again: the same bottom

The Path on the Loss Map

w_1 and w_4 , the other six frozen

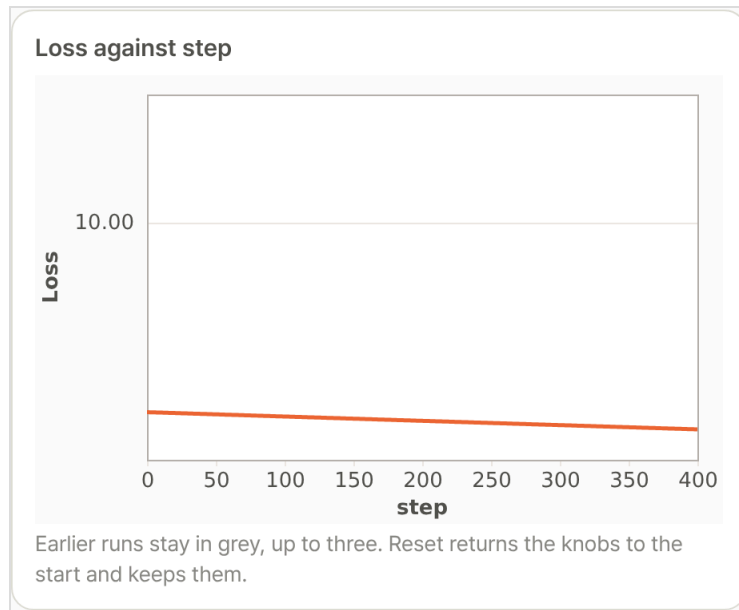


- The map from last time, with the path drawn on it
- Each step goes downhill from where it stands

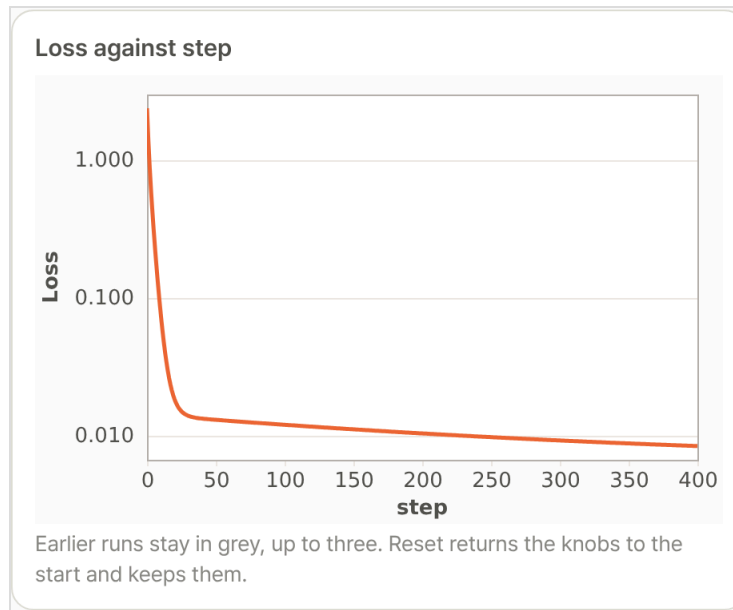
Learning rate

Three Step Sizes

$lr = 0.0002, 0.3, 0.6$



Too small. Safe, but slow.



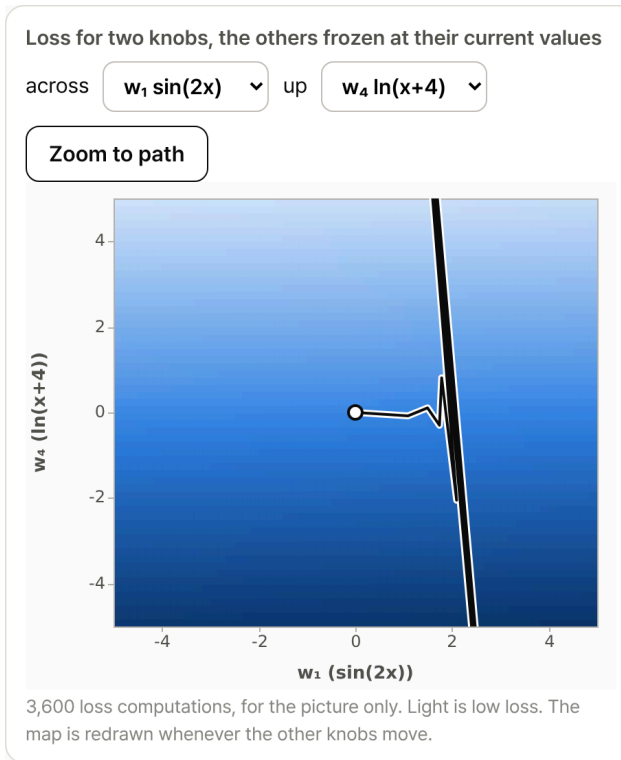
About right.



Too large. The loss goes up.

Too Large a Step

The path on the map



- The step jumps across the valley instead of walking down it
- Large enough, and every step lands higher than the last: the loss blows up

Where you start

Unknown Frequencies

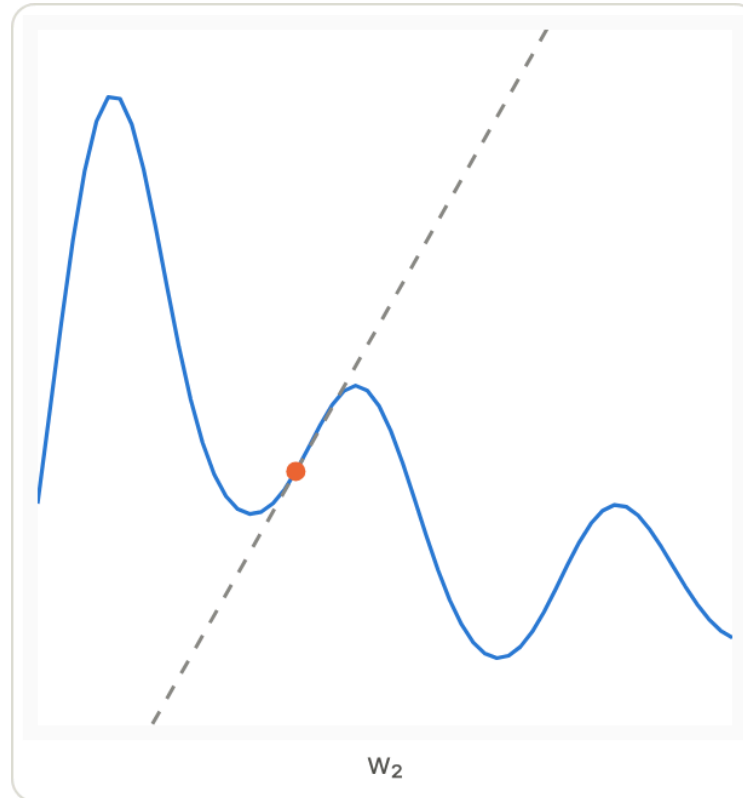
Twelve knobs

$$y = w_1 \sin(w_2 x + w_3) + w_4 \cos(w_5 x + w_6) + w_7 e^x / 20 + w_8 \ln(x+4) + w_9 (x/3)^4 + w_{10} (x/3)^2 + w_{11} (x/3) + w_{12}$$

- Same data as last time
- Last time the 2 in $\sin(2x)$ and the 3 in $\cos(3x)$ were given. Now they are knobs too

Loss Against a Frequency Knob

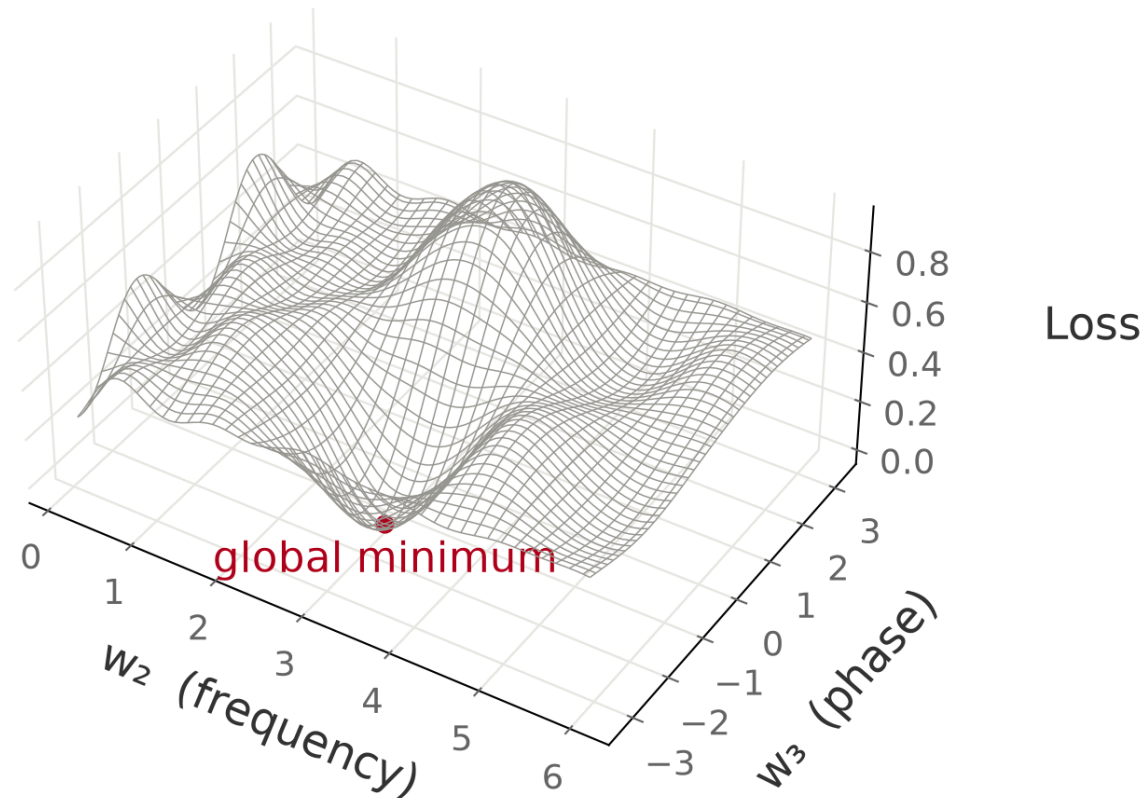
w_2 , the other eleven frozen



- Several valleys. The lowest is the global minimum, the others are local minima
- Every curve of the eight-knob model was a bowl with one minimum

Two Knobs at Once

w_2 and w_3 , the other ten frozen at their fitted values



- Every dip is a valley that the rule can settle into

Three Starts

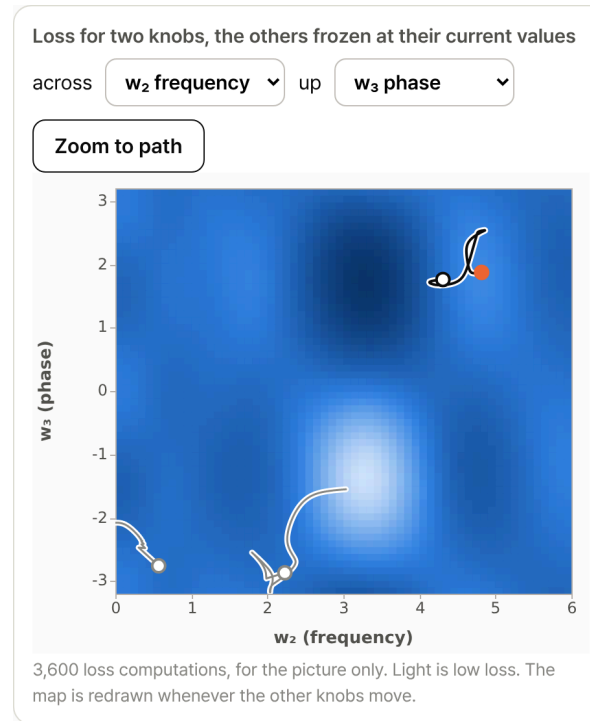
Demo, tab 4

Runs so far				
Run	Start	Steps	Final loss	
1	A	5000	0.006	Load
2	B	5000	0.033	Load
3	C	5000	0.107	Load

- Start A ends at loss 0.006, start B at 0.033, start C at 0.107
- Same data, same model, same rule, same learning rate

Local Minimum

Why the run stops there



- At the bottom of any valley every slope is near zero, so the rule stops
- Gradient descent sees only the slope where it stands
- Where you start decides which valley you end in

What we have

Three Parts of Machine Learning

- A model: a function with parameters, $y = f(x; w)$
- A loss: one number that measures how far the predictions are from the data
- A search: gradient descent. Measure the slopes, step against them, repeat
- Running the search on data is called training

The Same Rule at Any Size

Knobs k	One gradient by nudging	A grid with 100 values per knob
2	3 loss computations	10^4
8	9 loss computations	10^{16}
8 billion	8 billion and 1	100^8 billion

- The rule does not change from 8 knobs to 8 billion
- Nudging one knob at a time does not scale
- Week 5, backpropagation: every slope for the cost of a few loss computations
- Next time: a model built from neurons